

Benchmarking scMutrace and smartSeq

Qun Li

13 Aug 2025 at 14:07:57

Contents

1	Libraries	1
2	Functions	2
3	Load Data	3
4	Mutation burden	3
5	Benchmarking	13
6	Census genes celltypelevel	15
7	Census genes samplelevel	18
8	GermlinePopulation	21
9	sessionInfo	25

#ENV

```
rm(list = ls())  
setwd("/Users/liqun/Desktop/scMutrace-tutorial/ReproducibilityAnalysis/Figure3/")
```

1 Libraries

```
library(ggplot2)  
library(ggpubr)  
library(dplyr)  
##  
## Attaching package: 'dplyr'  
## The following objects are masked from 'package:stats':  
##  
## filter, lag
```

```
## The following objects are masked from 'package:base':
##
## intersect, setdiff, setequal, union
library(circlize)
## =====
## circlize version 0.4.16
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize\_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
## in R. Bioinformatics 2014.
##
## This message can be suppressed by:
## suppressPackageStartupMessages(library(circlize))
## =====
library(ggVennDiagram)
library(ComplexHeatmap)
## Loading required package: grid
## =====
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
## genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
## suppressPackageStartupMessages(library(ComplexHeatmap))
## =====
```

2 Functions

```
retriveCellsByLocation <- function(CSM, locList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[, c("CellTag",locList)]
  data_MutPerCell_CSM_tmp <- data.frame(
    CellTag = rownames(CSM_tmp),
    MutNum = rowSums(CSM_tmp == 1),
    WTNNum = rowSums(CSM_tmp == 0)
  )
  cells_Total <- data_MutPerCell_CSM_tmp[rowSums(data_MutPerCell_CSM_tmp[,c("MutNum", "WTNNum")]) > 0, ]
  cells_WT <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "WTNNum"] > 0,]$CellTag
```

```

cells_Mut <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "MutNum"] > 0,]$CellTag
cells <- list(Total = cells_Total, Mut = cells_Mut, WT = cells_WT)
return(cells)
}

retriveLocsByCells <- function(CSM, cellList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[CSM$CellTag %in% cellList,]
  data_CellPerMut_CSM_tmp <- data.frame(
    Location = colnames(CSM_tmp),
    CellMutNum = colSums(CSM_tmp == 1),
    CellWTNum = colSums(CSM_tmp == 0)
  )
  pos_Total <- data_CellPerMut_CSM_tmp[rowSums(data_CellPerMut_CSM_tmp[, c("CellMutNum", "CellWTNum")])
  pos_Mut <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellMutNum"] > 0,]$Location
  pos_WT <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellWTNum"] > 0,]$Location
  pos <- list(Total = pos_Total, Mut = pos_Mut, WT = pos_WT)
  return(pos)
}

```

3 Load Data

```

data_CSM <- readRDS("../Data/HumanNSCLC/data_CSM_NSCLC_SMARTSeq.rds")
data_CSM_TH179 <- data_CSM$TH179
data_CSM_TH179NAT <- data_CSM$TH179NAT
data_CSM_TH248 <- data_CSM$TH248

data_SSM <- readRDS("../Data/HumanNSCLC/data_SSM_NSCLC_SMARTSeq.rds")
data_SSM_TH179_ann <- data_SSM$TH179
data_SSM_TH179NAT_ann <- data_SSM$TH179NAT
data_SSM_TH248_ann <- data_SSM$TH248

data_meta <- read.table("../Data/HumanNSCLC/SMARTSeq_NSCLC_meta.txt", header = T, sep = "\t")

```

4 Mutation burden

```

# TH179
data_MutPerCell_TH179 <- data.frame(
  CellTag = rownames(data_CSM_TH179),
  MutNum_somatic = rowSums(data_CSM_TH179 == 1),
  WTNum_somatic = rowSums(data_CSM_TH179 == 0)
)

data_CellPerMut_TH179 <- data.frame(
  Location = colnames(data_CSM_TH179),
  CellMutNum = colSums(data_CSM_TH179 == 1),
  CellWTNum = colSums(data_CSM_TH179 == 0)
)

```

```

data_MutPerCell_TH179_ann <- merge(data_MutPerCell_TH179, data_meta, by = "CellTag")

# TH179NAT
data_MutPerCell_TH179NAT <- data.frame(
  CellTag = rownames(data_CSM_TH179NAT),
  MutNum_somatic = rowSums(data_CSM_TH179NAT == 1),
  WTNum_somatic = rowSums(data_CSM_TH179NAT == 0)
)

data_CellPerMut_TH179NAT <- data.frame(
  Location = colnames(data_CSM_TH179NAT),
  CellMutNum = colSums(data_CSM_TH179NAT == 1),
  CellWTNum = colSums(data_CSM_TH179NAT == 0)
)

data_MutPerCell_TH179NAT_ann <- merge(data_MutPerCell_TH179NAT, data_meta, by = "CellTag")

# TH248
data_MutPerCell_TH248 <- data.frame(
  CellTag = rownames(data_CSM_TH248),
  MutNum_somatic = rowSums(data_CSM_TH248 == 1),
  WTNum_somatic = rowSums(data_CSM_TH248 == 0)
)

data_CellPerMut_TH248 <- data.frame(
  Location = colnames(data_CSM_TH248),
  CellMutNum = colSums(data_CSM_TH248 == 1),
  CellWTNum = colSums(data_CSM_TH248 == 0)
)

data_MutPerCell_TH248_ann <- merge(data_MutPerCell_TH248, data_meta, by = "CellTag")

# comparison
my_comparisons_0 <- list(c("cancer", "noncancer"))

my_comparisons_00 <- list(c("cancer", "noncancer"),
  c("cancer", "normal"))

my_comparisons_1 <- list(c("LT_S58_cancer", "LT_S58_noncancer"),
  c("LT_S79_cancer", "LT_S79_noncancer"),
  c("LT_S80_cancer", "LT_S80_noncancer"))

my_comparisons_11 <- list(c("LT_S58_cancer", "LT_S58_noncancer"),
  c("LT_S79_cancer", "LT_S79_noncancer"),
  c("LT_S80_cancer", "LT_S80_noncancer"),
  c("LT_S58_cancer", "LT_S78_normal"),
  c("LT_S79_cancer", "LT_S78_normal"),
  c("LT_S80_cancer", "LT_S78_normal"),
  c("LT_S58_noncancer", "LT_S78_normal"),
  c("LT_S79_noncancer", "LT_S78_normal"),
  c("LT_S80_noncancer", "LT_S78_normal"))

```

```

my_comparisons_2 <- list(c("LT_S69_cancer", "LT_S69_noncancer"),
                        c("LT_S74_cancer", "LT_S74_noncancer"))

my_comparisons_3 <- list(c("LT_S58_cancer", "LT_S58_noncancer"),
                        c("LT_S79_cancer", "LT_S79_noncancer"),
                        c("LT_S80_cancer", "LT_S80_noncancer"),
                        c("LT_S58_cancer", "LT_S78_normal"),
                        c("LT_S79_cancer", "LT_S78_normal"),
                        c("LT_S80_cancer", "LT_S78_normal"),
                        c("LT_S58_noncancer", "LT_S78_normal"),
                        c("LT_S79_noncancer", "LT_S78_normal"),
                        c("LT_S80_noncancer", "LT_S78_normal"),
                        c("LT_S69_cancer", "LT_S69_noncancer"),
                        c("LT_S74_cancer", "LT_S74_noncancer"))

data_MutPerCell_TH179_TH179NAT_ann <- rbind(data_MutPerCell_TH179_ann, data_MutPerCell_TH179NAT_ann)
data_MutPerCell_TH179_TH179NAT_ann$type <- factor(data_MutPerCell_TH179_TH179NAT_ann$type, levels = c("cancer", "noncancer"))
data_MutPerCell_TH179_TH179NAT_ann$sampleCellType <- factor(data_MutPerCell_TH179_TH179NAT_ann$sampleCellType, levels = c("cancer", "noncancer"))

p_TH179_cancer_noncancer <- ggplot(data_MutPerCell_TH179_ann, aes(x=as.factor(type), y=MutNum_somatic, fill=type)) +
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("cancer" = "#ca0020",
                              "noncancer" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_0,
                    method = "wilcox.test",
                    label = "p.format")

p_TH179_TH179NAT_cancer_noncancer <- ggplot(data_MutPerCell_TH179_TH179NAT_ann, aes(x=as.factor(type), y=MutNum_somatic, fill=type)) +
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("cancer" = "#ca0020",
                              "noncancer" = "#d9d9d9",
                              "normal" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_00,
                    method = "wilcox.test",
                    label = "p.format")

p_TH248_cancer_noncancer <- ggplot(data_MutPerCell_TH248_ann, aes(x=as.factor(type), y=MutNum_somatic, fill=type)) +
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("cancer" = "#ca0020",
                              "noncancer" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_0,
                    method = "wilcox.test",

```

```

        label = "p.format")

p_TH179_sampleCellType <- ggplot(data_MutPerCell_TH179_ann, aes(x=as.factor(sampleCellType), y=MutNum_s
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("LT_S58_cancer" = "#ca0020",
                                "LT_S58_noncancer" = "#d9d9d9",
                                "LT_S79_cancer" = "#ca0020",
                                "LT_S79_noncancer" = "#d9d9d9",
                                "LT_S80_cancer" = "#ca0020",
                                "LT_S80_noncancer" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_1,
                    method = "wilcox.test",
                    label = "p.format")

p_TH179_TH179NAT_sampleCellType <- ggplot(data_MutPerCell_TH179_TH179NAT_ann, aes(x=as.factor(sampleCel
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("LT_S58_cancer" = "#ca0020",
                                "LT_S58_noncancer" = "#d9d9d9",
                                "LT_S79_cancer" = "#ca0020",
                                "LT_S79_noncancer" = "#d9d9d9",
                                "LT_S80_cancer" = "#ca0020",
                                "LT_S80_noncancer" = "#d9d9d9",
                                "LT_S78_normal" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_11,
                    method = "wilcox.test",
                    label = "p.format")

p_TH248_sampleCellType <- ggplot(data_MutPerCell_TH248_ann, aes(x=as.factor(sampleCellType), y=MutNum_s
  geom_boxplot(outliers = FALSE) +
  geom_jitter(width=0.2, size=1, alpha=0.5) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("LT_S69_cancer" = "#ca0020",
                                "LT_S69_noncancer" = "#d9d9d9",
                                "LT_S74_cancer" = "#ca0020",
                                "LT_S74_noncancer" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_2,
                    method = "wilcox.test",
                    label = "p.format")

data_MutPerCell_TH248_TH179_TH179NAT_ann_ <- rbind(data_MutPerCell_TH248_ann, data_MutPerCell_TH179_TH1
data_MutPerCell_TH248_TH179_TH179NAT_ann_$sampleCellType <- factor(data_MutPerCell_TH248_TH179_TH179NAT
                                levels = c("LT_S74_cancer", "LT_S74_n
                                                "LT_S69_cancer", "LT_S69_n
                                                "LT_S58_cancer", "LT_S58_n
                                                "LT_S79_cancer", "LT_S79_n

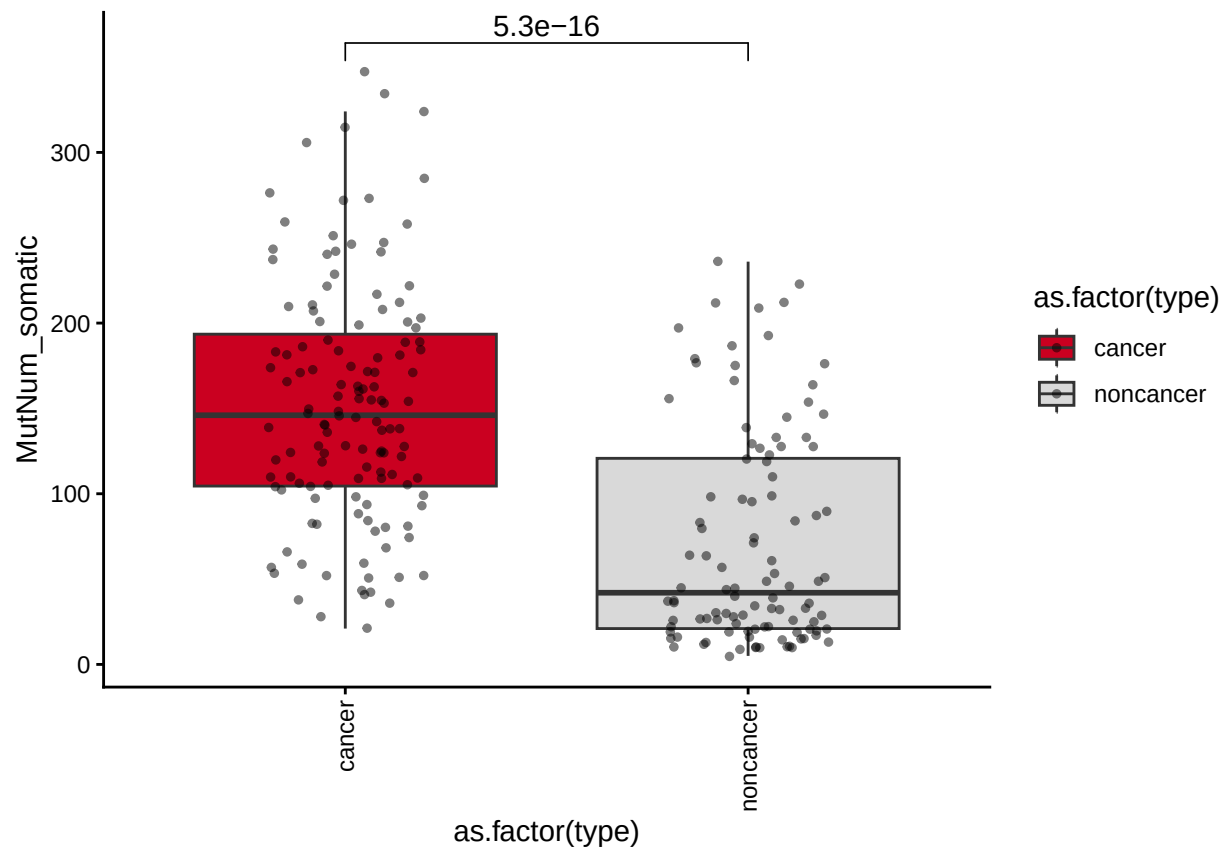
```

```

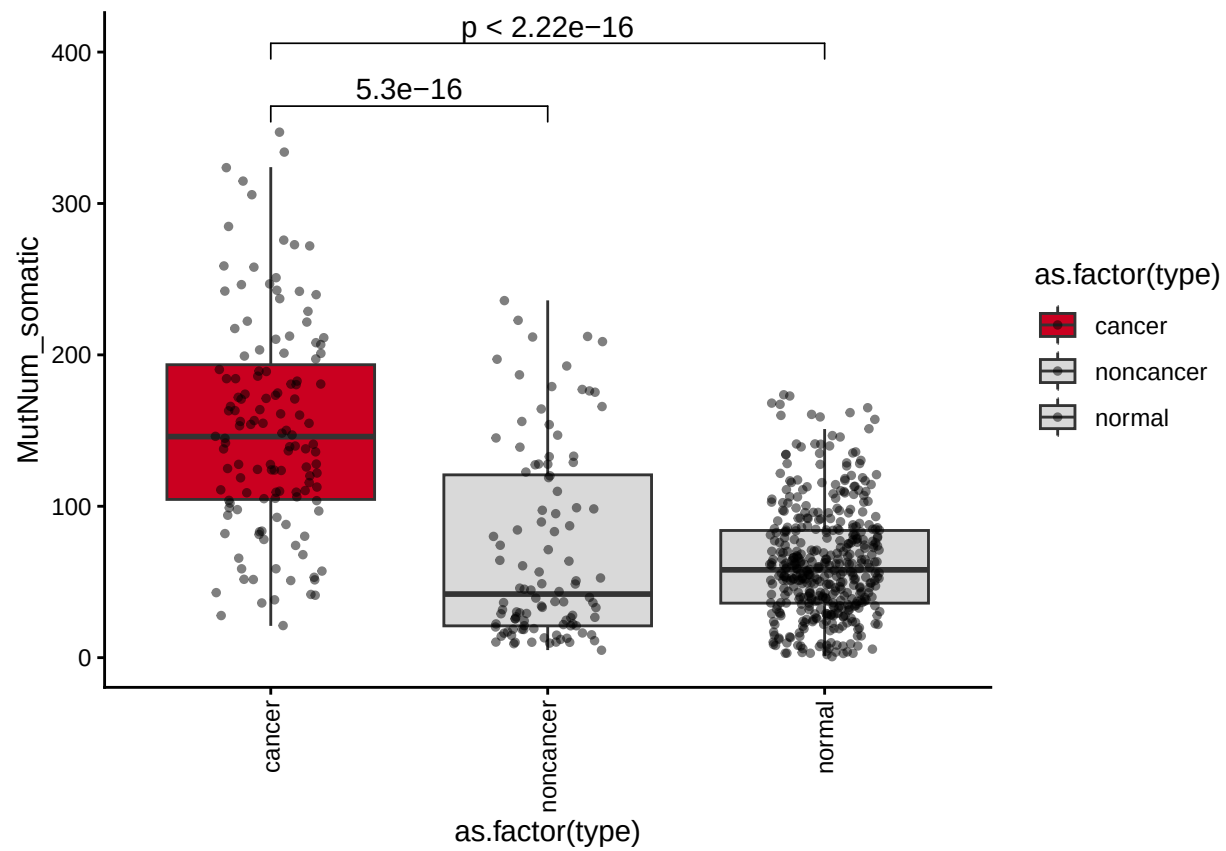
"LT_S80_cancer", "LT_S80_1
"LT_S78_normal"))
p_TH248_TH179_TH179NAT_sampleCellType <- ggplot(data_MutPerCell_TH248_TH179_TH179NAT_ann_, aes(x=as.factor(
  geom_boxplot(outliers = FALSE) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  scale_fill_manual(values = c("LT_S69_cancer" = "#ca0020",
                                "LT_S69_noncancer" = "#d9d9d9",
                                "LT_S74_cancer" = "#ca0020",
                                "LT_S74_noncancer" = "#d9d9d9",
                                "LT_S58_cancer" = "#ca0020",
                                "LT_S58_noncancer" = "#d9d9d9",
                                "LT_S79_cancer" = "#ca0020",
                                "LT_S79_noncancer" = "#d9d9d9",
                                "LT_S80_cancer" = "#ca0020",
                                "LT_S80_noncancer" = "#d9d9d9",
                                "LT_S78_normal" = "#d9d9d9")) +
  stat_compare_means(comparisons = my_comparisons_3,
                     method = "wilcox.test",
                     label = "p.format")

p_TH179_cancer_noncancer

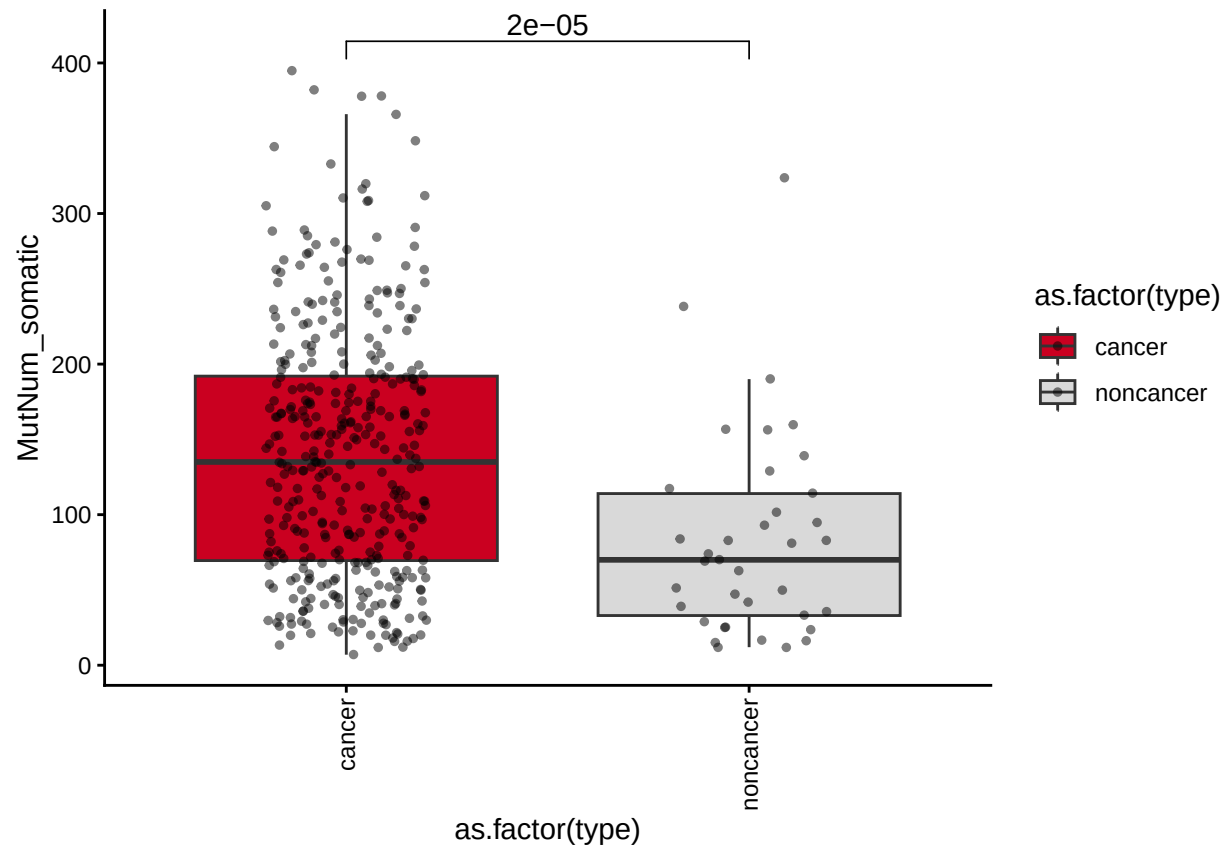
```



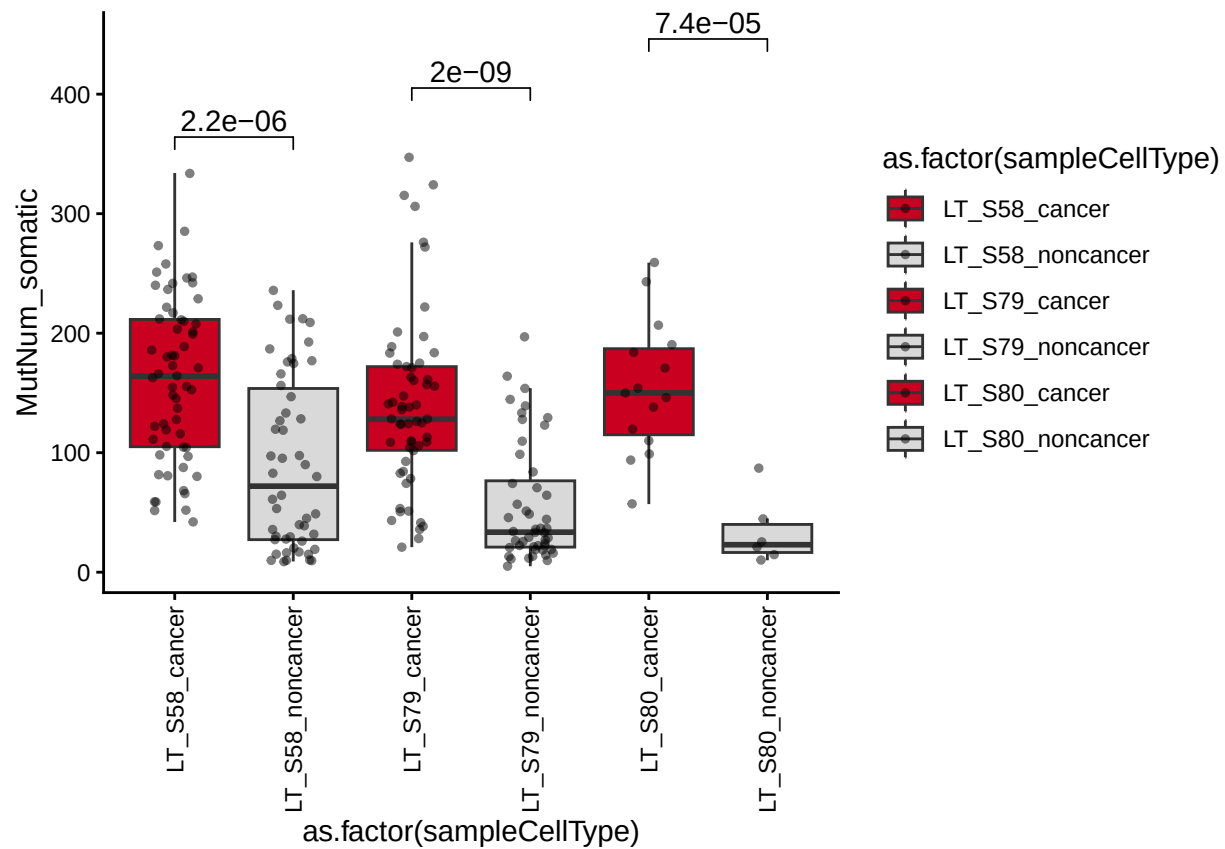
```
p_TH179_TH179NAT_cancer_noncancer
```



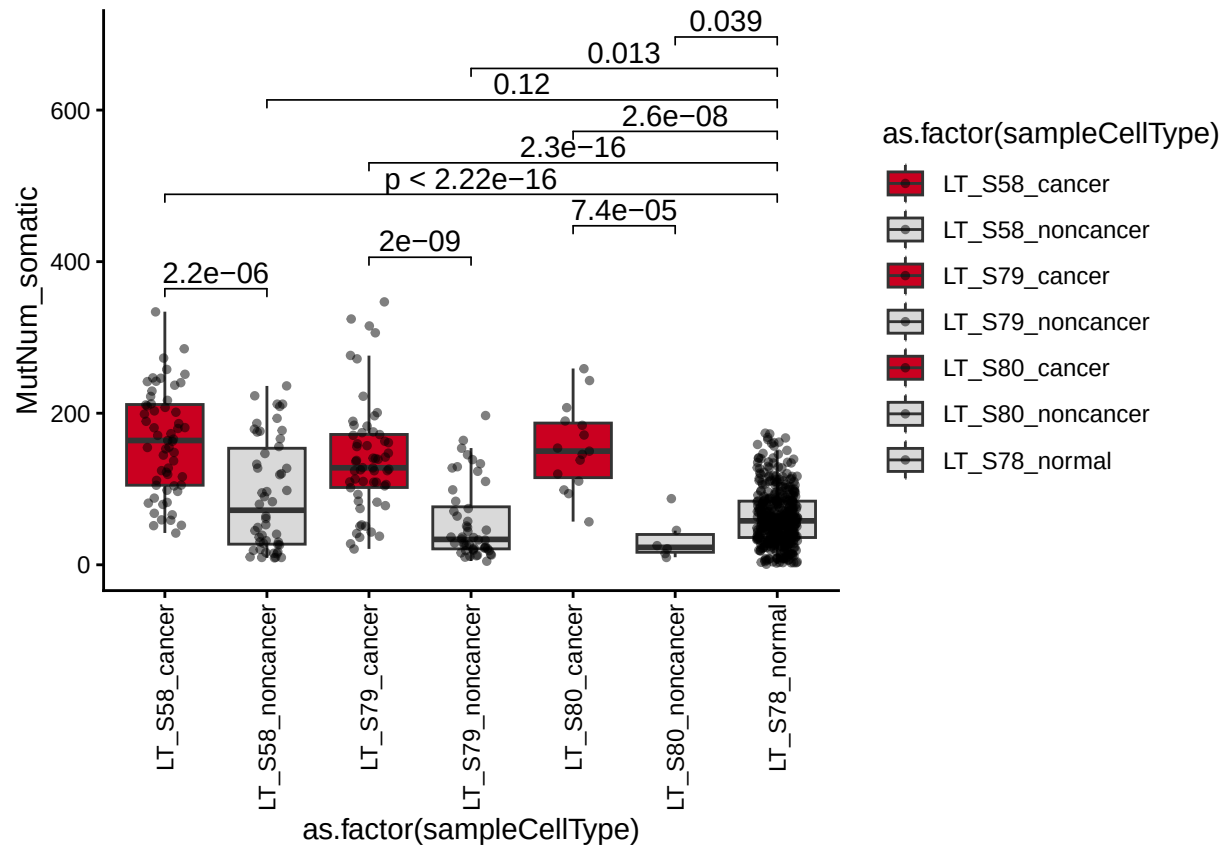
p_TH248_cancer_noncancer



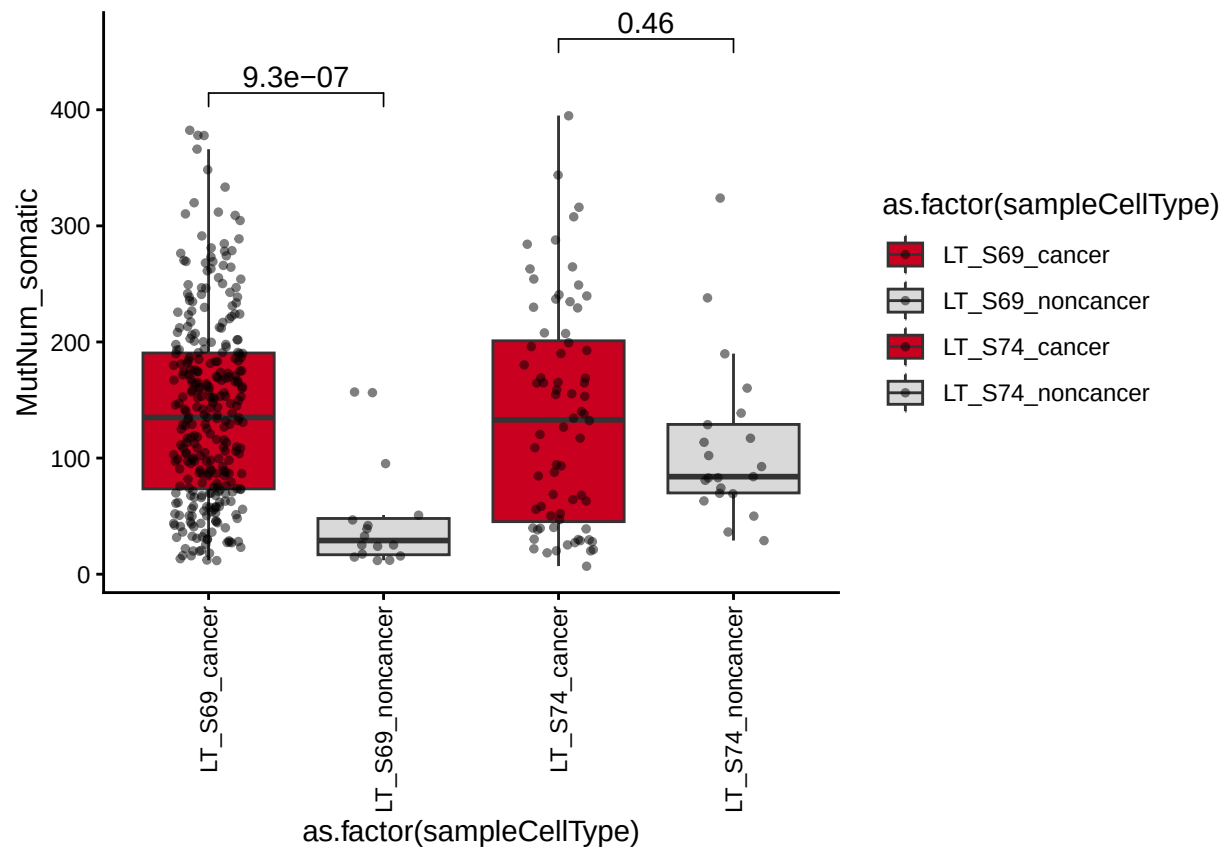
p_TH179_sampleCellType



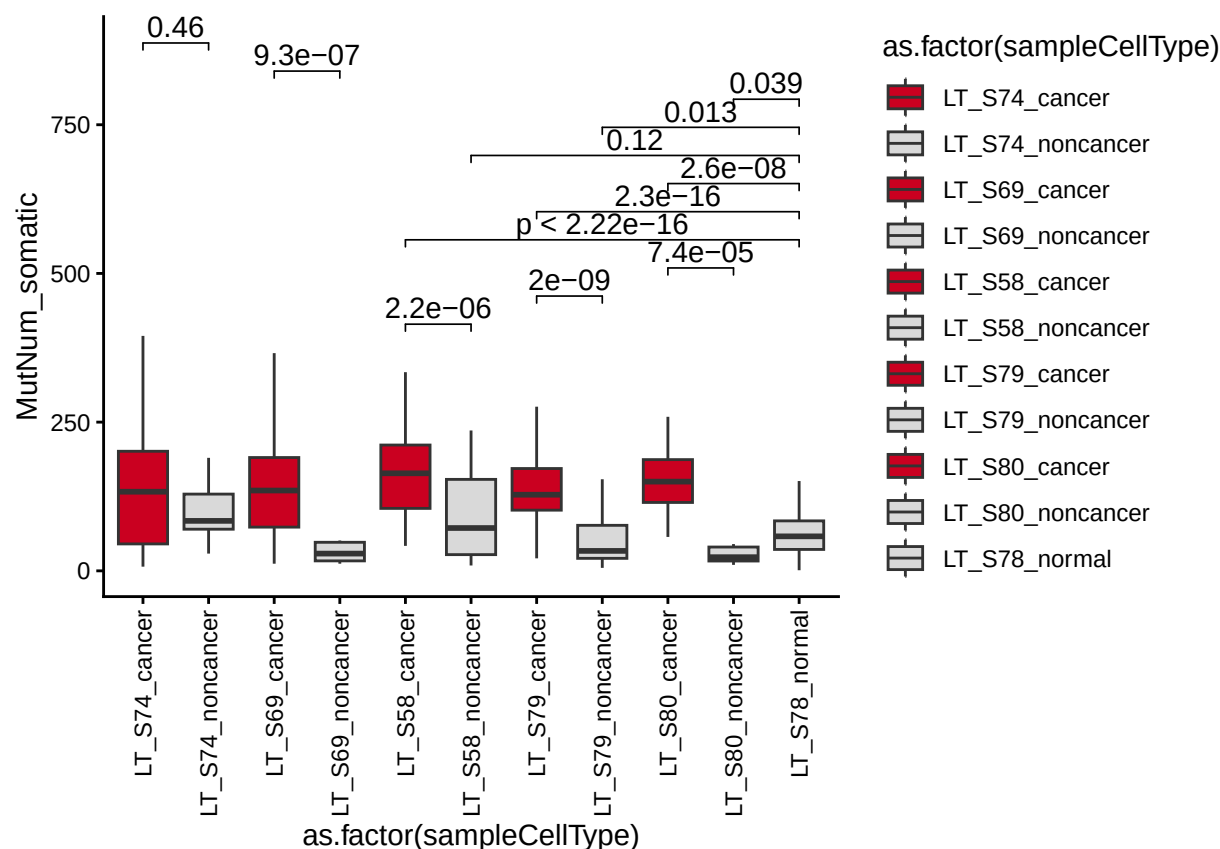
p_TH179_TH179NAT_sampleCellType



p_TH248_sampleCellType



p_TH248_TH179_TH179NAT_sampleCellType



5 Benchmarking

```
data_benchmarking <- read.table("../Data/HumanNSCLC/data_benchmarking_NSCLC.txt", sep = "\t", header = TRUE)
data_benchmarking_meta <- merge(data_benchmarking, data_meta, by = "CellTag")
data_benchmarking_meta$Gene <- sub("_.*", "", data_benchmarking_meta$Mutation)

mutations_benchmarking <- c("BRAF_V600E", "BRCA2_N372H", "DROSHA_S321L", "EGFR_K745_A750_T", "ROS1_K222R")
col_benchmarking <- c("Mutation", "Precision", "Recall", "F1Score")

data_benchmarking_res <- as.data.frame(matrix(data = 0, nrow = length(mutations_benchmarking), ncol = length(col_benchmarking)))
colnames(data_benchmarking_res) <- col_benchmarking

for (i in 1:length(mutations_benchmarking)){
  mut <- mutations_benchmarking[i]
  cell_smart <- data_benchmarking_meta[data_benchmarking_meta$Mutation == mut & data_benchmarking_meta$CellType == "cancer", ]
  cell_scmut <- data_benchmarking_meta[data_benchmarking_meta$Mutation == mut & data_benchmarking_meta$CellType == "noncancer", ]
  cell_commo <- intersect(cell_smart, cell_scmut)
  precision <- length(cell_commo) / length(cell_scmut)
  recall <- length(cell_commo) / length(cell_smart)
  f1score <- (2 * precision * recall) / (precision + recall)
  data_benchmarking_res[i,]$Mutation <- mut
  data_benchmarking_res[i,]$Precision <- precision
  data_benchmarking_res[i,]$Recall <- recall
}
```

```

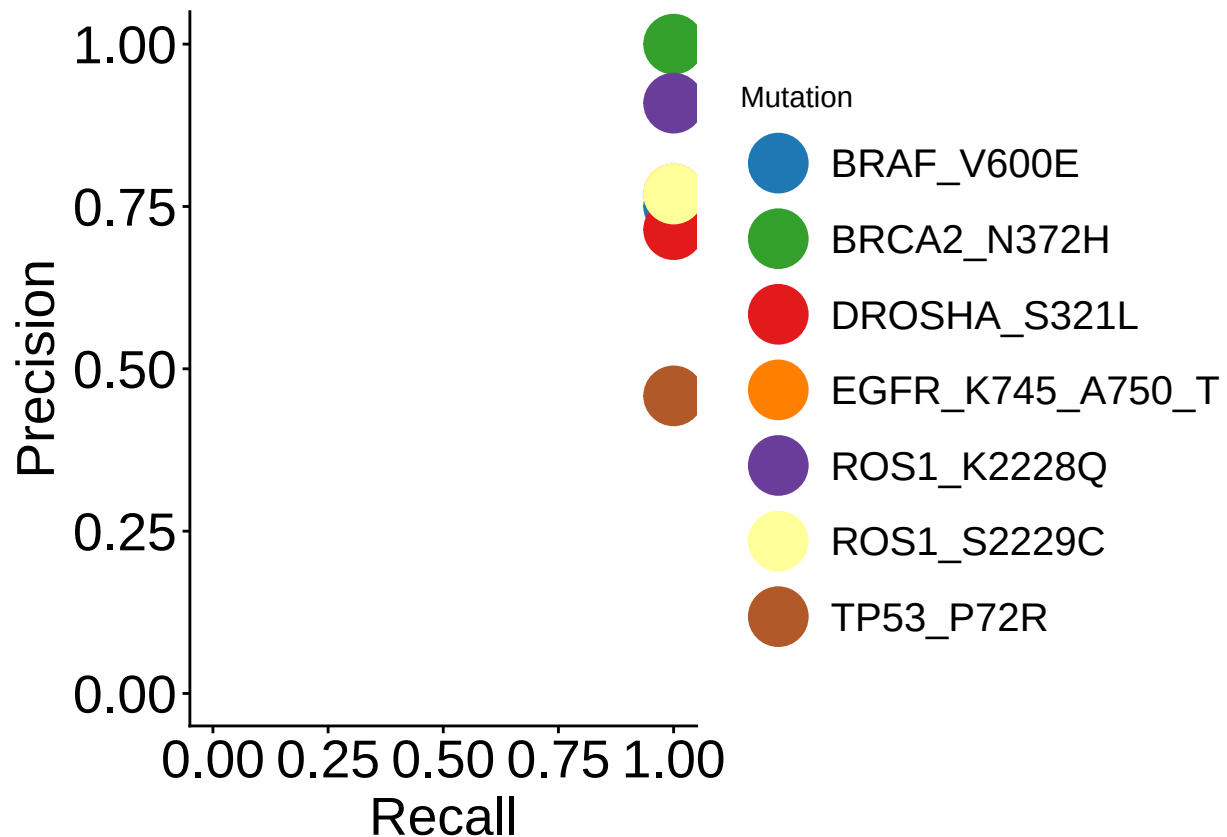
data_benchmarking_res[i,]$F1Score <- f1score
}

genesColor <- c(BRAF_V600E = "#1f78b4",
  BRCA2_N372H = "#33a02c",
  DROSHA_S321L = "#e31a1c",
  EGFR_K745_A750_T = "#ff7f00",
  ROS1_K2228Q = "#6a3d9a",
  ROS1_S2229C = "#ffff99",
  TP53_P72R = "#b15928")

p_benchmarking <- ggplot(data = data_benchmarking_res) + geom_point(mapping = aes(x = Recall , y = Precision),
  scale_color_manual(values = genesColor) +
  xlab("Recall") + ylab("Precision") +
  xlim(0, 1) +
  ylim(0, 1) +
  theme_classic() +
  theme(axis.title = element_text(size=20), axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=20), legend.text=element_text(size=15))

p_benchmarking

```

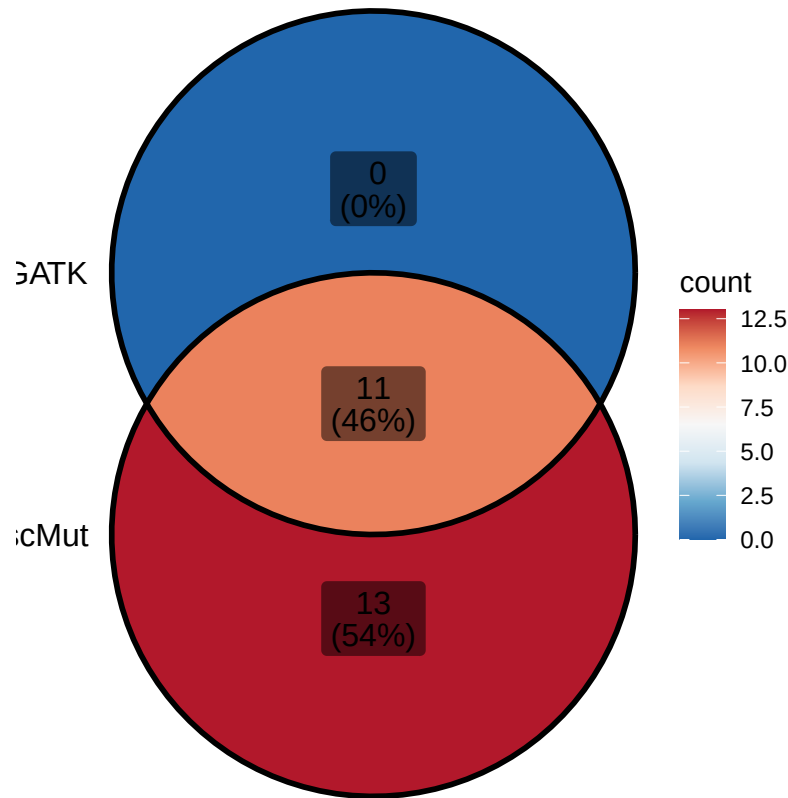


```

venn_list_TP53_P72R <- list(scMut = unique(data_benchmarking_meta %>% filter(Mutation == "TP53_P72R" & T
  GATK = unique(data_benchmarking_meta %>% filter(Mutation == "TP53_P72R" & T

```

```
p_venn_TP53_P72R <- ggVennDiagram(venn_list_TP53_P72R, category.names = c("scMut", "GATK")) + scale_fill_manual(values = c("#1f77b4", "#d62728"))
p_venn_TP53_P72R
```



6 Census genes celltypelevel

```
# from COSMIC database
# TP53 from original paper
data_censusgenes <- c("AKT1", "ALK", "BAP1", "BRAF", "CCDC6", "CD74", "DDR2",
  "DROSHA", "EGFR", "EML4", "ERBB2", "ERBB4", "EZR",
  "FGFR2", "GOLPH3", "HIP1", "KDR", "KEAP1", "KIF5B",
  "LRIG3", "MAP2K1", "MAP2K2", "NFE2L2", "NKX2-1",
  "NRG1", "PIK3CB", "PTPN13", "RET", "ROS1", "SDC4",
  "SMARCA4", "SOX2", "STK11", "TFG", "TPM3", "TPR", "TP53")

data_SSM_TH179_ann_selected <- data_SSM_TH179_ann[data_SSM_TH179_ann$Gene.refGene %in% data_censusgenes]
data_SSM_TH179NAT_ann_selected <- data_SSM_TH179NAT_ann[data_SSM_TH179NAT_ann$Gene.refGene %in% data_censusgenes]
data_SSM_TH248_ann_selected <- data_SSM_TH248_ann[data_SSM_TH248_ann$Gene.refGene %in% data_censusgenes]

census_genes <- unique(c(data_SSM_TH179_ann_selected$Gene.refGene, data_SSM_TH248_ann_selected$Gene.refGene))

# TH179
cells_LT_S58_cancer <- data_meta[data_meta$sample_name == "LT_S58" & data_meta$type == "cancer",]$CellType
```

```

cells_LT_S58_noncancer <- data_meta[data_meta$sample_name == "LT_S58" & data_meta$type == "noncancer",]
cells_LT_S79_cancer <- data_meta[data_meta$sample_name == "LT_S79" & data_meta$type == "cancer",]$CellID
cells_LT_S79_noncancer <- data_meta[data_meta$sample_name == "LT_S79" & data_meta$type == "noncancer",]
cells_LT_S80_cancer <- data_meta[data_meta$sample_name == "LT_S80" & data_meta$type == "cancer",]$CellID
cells_LT_S80_noncancer <- data_meta[data_meta$sample_name == "LT_S80" & data_meta$type == "noncancer",]

# TH248
cells_LT_S69_cancer <- data_meta[data_meta$sample_name == "LT_S69" & data_meta$type == "cancer",]$CellID
cells_LT_S69_noncancer <- data_meta[data_meta$sample_name == "LT_S69" & data_meta$type == "noncancer",]
cells_LT_S74_cancer <- data_meta[data_meta$sample_name == "LT_S74" & data_meta$type == "cancer",]$CellID
cells_LT_S74_noncancer <- data_meta[data_meta$sample_name == "LT_S74" & data_meta$type == "noncancer",]

# TH249NAT
cells_TH179_NAT_normal <- data_meta[data_meta$patient_id == "TH179_NAT" & data_meta$type == "normal",]$CellID

col_fra <- c("LT_S58_cancer", "LT_S58_noncancer", "LT_S79_cancer", "LT_S79_noncancer",
            "LT_S80_cancer", "LT_S80_noncancer", "LT_S69_cancer", "LT_S69_noncancer",
            "LT_S74_cancer", "LT_S74_noncancer", "TH179_NAT")

data_census_fraction <- as.data.frame(matrix(data = 0, nrow = length(census_genes), ncol = length(col_fra),
rownames(data_census_fraction) <- census_genes
colnames(data_census_fraction) <- col_fra

genes_TH179 <- unique(data_SSM_TH179_ann_selected$Gene.refGene)
genes_TH179NAT <- unique(data_SSM_TH179NAT_ann_selected$Gene.refGene)
genes_TH248 <- unique(data_SSM_TH248_ann_selected$Gene.refGene)

for (i in seq_along(census_genes)) {
  gene <- census_genes[i]
  #data_census_fraction[i, "gene"] <- gene

  # TH179
  if (gene %in% genes_TH179) {
    sites_TH179 <- data_SSM_TH179_ann_selected[data_SSM_TH179_ann_selected$Gene.refGene == gene, "Mutations"]
    cells_TH179 <- retrieveCellsByLocation(data_CSM_TH179, sites_TH179)$Mut

    data_census_fraction[i, "LT_S58_cancer"] <- length(intersect(cells_LT_S58_cancer, cells_TH179))
    data_census_fraction[i, "LT_S58_noncancer"] <- length(intersect(cells_LT_S58_noncancer, cells_TH179))
    data_census_fraction[i, "LT_S79_cancer"] <- length(intersect(cells_LT_S79_cancer, cells_TH179))
    data_census_fraction[i, "LT_S79_noncancer"] <- length(intersect(cells_LT_S79_noncancer, cells_TH179))
    data_census_fraction[i, "LT_S80_cancer"] <- length(intersect(cells_LT_S80_cancer, cells_TH179))
    data_census_fraction[i, "LT_S80_noncancer"] <- length(intersect(cells_LT_S80_noncancer, cells_TH179))
  }

  # TH248
  if (gene %in% genes_TH248) {
    sites_TH248 <- data_SSM_TH248_ann_selected[data_SSM_TH248_ann_selected$Gene.refGene == gene, "Mutations"]
    cells_TH248 <- retrieveCellsByLocation(data_CSM_TH248, sites_TH248)$Mut

    data_census_fraction[i, "LT_S69_cancer"] <- length(intersect(cells_LT_S69_cancer, cells_TH248))
    data_census_fraction[i, "LT_S69_noncancer"] <- length(intersect(cells_LT_S69_noncancer, cells_TH248))
    data_census_fraction[i, "LT_S74_cancer"] <- length(intersect(cells_LT_S74_cancer, cells_TH248))
  }
}

```

```

    data_census_fraction[i, "LT_S74_noncancer"] <- length(intersect(cells_LT_S74_noncancer, cells_TH248
  }

  # TH179_NAT
  if (gene %in% genes_TH179NAT) {
    sites_TH179NAT <- data_SSM_TH179NAT_ann_selected[data_SSM_TH179NAT_ann_selected$Gene.refGene == gene
    cells_TH179NAT <- retriveCellsByLocation(data_CSM_TH179NAT, sites_TH179NAT)$Mut

    data_census_fraction[i, "TH179_NAT"] <- length(intersect(cells_TH179_NAT_normal, cells_TH179NAT)) /
  }
}

max_val <- max(data_census_fraction)

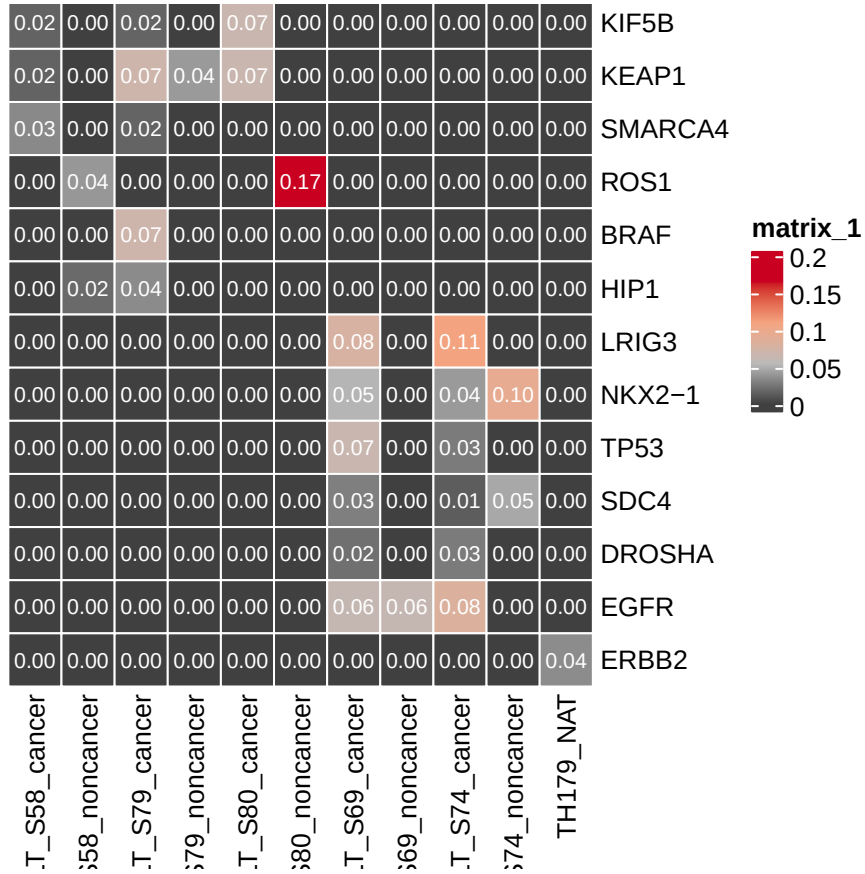
col_fun_v1 <- colorRamp2(c(0, max_val * 0.33, max_val * 0.66, max_val), c("#404040", "#bababa", "#f4a585", "#f4a585"))

p_proportion <- pheatmap(data_census_fraction,
                          cluster_rows = FALSE,
                          cluster_cols = FALSE,
                          cellwidth = 20,
                          cellheight = 20,
                          display_numbers = TRUE,
                          number_color = "white",
                          border_color = "white",
                          color = col_fun_v1)

## Warning: The input is a data frame, convert it to the matrix.

p_proportion

```



7 Census genes samplelevel

```
# from COSMIC database
# TP53 from original paper
data_censusgenes <- c("AKT1", "ALK", "BAP1", "BRAF", "CCDC6", "CD74", "DDR2",
  "DROSHA", "EGFR", "EML4", "ERBB2", "ERBB4", "EZR",
  "FGFR2", "GOLPH3", "HIP1", "KDR", "KEAP1", "KIF5B",
  "LRIG3", "MAP2K1", "MAP2K2", "NFE2L2", "NKX2-1",
  "NRG1", "PIK3CB", "PTPN13", "RET", "ROS1", "SDC4",
  "SMARCA4", "SOX2", "STK11", "TFG", "TPM3", "TPR", "TP53")

data_SSM_TH179_ann_selected <- data_SSM_TH179_ann[data_SSM_TH179_ann$Gene.refGene %in% data_censusgenes]
data_SSM_TH179NAT_ann_selected <- data_SSM_TH179NAT_ann[data_SSM_TH179NAT_ann$Gene.refGene %in% data_censusgenes]
data_SSM_TH248_ann_selected <- data_SSM_TH248_ann[data_SSM_TH248_ann$Gene.refGene %in% data_censusgenes]

census_genes <- unique(c(data_SSM_TH179_ann_selected$Gene.refGene, data_SSM_TH248_ann_selected$Gene.refGene))

# TH179
cells_LT_S58 <- data_meta[data_meta$sample_name == "LT_S58",]$CellTag
cells_LT_S79 <- data_meta[data_meta$sample_name == "LT_S79",]$CellTag
cells_LT_S80 <- data_meta[data_meta$sample_name == "LT_S80",]$CellTag
```

```

# TH248
cells_LT_S69 <- data_meta[data_meta$sample_name == "LT_S69",]$CellTag
cells_LT_S74 <- data_meta[data_meta$sample_name == "LT_S74",]$CellTag

# TH249NAT
cells_TH179_NAT_normal <- data_meta[data_meta$patient_id == "TH179_NAT",]$CellTag

col_fra <- c("S74_GATK", "S69_GATK",
            "S58_GATK", "S79_GATK",
            "S80_GATK", "NAT_GATK",
            "S74", "S69",
            "S58", "S79",
            "S80", "NAT")

data_census_fraction_sample <- as.data.frame(matrix(data = 0, nrow = length(census_genes), ncol = length(col_fra)))
rownames(data_census_fraction_sample) <- census_genes
colnames(data_census_fraction_sample) <- col_fra

genes_TH179 <- unique(data_SSM_TH179_ann_selected$Gene.refGene)
genes_TH179NAT <- unique(data_SSM_TH179NAT_ann_selected$Gene.refGene)
genes_TH248 <- unique(data_SSM_TH248_ann_selected$Gene.refGene)

for (i in seq_along(census_genes)) {
  gene <- census_genes[i]

  # TH179
  if (gene %in% genes_TH179) {
    sites_TH179 <- data_SSM_TH179_ann_selected[data_SSM_TH179_ann_selected$Gene.refGene == gene, "Mutat"]
    cells_TH179 <- retriveCellsByLocation(data_CSM_TH179, sites_TH179)$Mut
    cells_TH179_GATK <- data_benchmarking_meta %>% filter(Type == "smartSeq" & patient_id == "TH179" & (
      data_census_fraction_sample[i, "S58"] <- length(intersect(cells_LT_S58, cells_TH179)) / length(c
      data_census_fraction_sample[i, "S79"] <- length(intersect(cells_LT_S79, cells_TH179)) / length(c
      data_census_fraction_sample[i, "S80"] <- length(intersect(cells_LT_S80, cells_TH179)) / length(c

      if (length(cells_TH179_GATK) > 0){
        data_census_fraction_sample[i, "S58_GATK"] <- length(intersect(cells_LT_S58, cells_TH179_GATK)) / length(c
        data_census_fraction_sample[i, "S79_GATK"] <- length(intersect(cells_LT_S79, cells_TH179_GATK)) / length(c
        data_census_fraction_sample[i, "S80_GATK"] <- length(intersect(cells_LT_S80, cells_TH179_GATK)) / length(c
      }
    }

  # TH248
  if (gene %in% genes_TH248) {
    sites_TH248 <- data_SSM_TH248_ann_selected[data_SSM_TH248_ann_selected$Gene.refGene == gene, "Mutat"]
    cells_TH248 <- retriveCellsByLocation(data_CSM_TH248, sites_TH248)$Mut
    cells_TH248_GATK <- data_benchmarking_meta %>% filter(Type == "smartSeq" & patient_id == "TH248" & (
      data_census_fraction_sample[i, "S69"] <- length(intersect(cells_LT_S69, cells_TH248)) / length(c
      data_census_fraction_sample[i, "S74"] <- length(intersect(cells_LT_S74, cells_TH248)) / length(c

      if (length(cells_TH248_GATK) > 0){
        data_census_fraction_sample[i, "S69_GATK"] <- length(intersect(cells_LT_S69, cells_TH248_GATK)) / length(c

```

```

    data_census_fraction_sample[i, "S74_GATK"] <- length(intersect(cells_LT_S74, cells_TH248_GATK))
  }
}

# TH179_NAT
if (gene %in% genes_TH179NAT) {
  sites_TH179NAT <- data_SSM_TH179NAT_ann_selected[data_SSM_TH179NAT_ann_selected$Gene.refGene == gene]
  cells_TH179NAT <- retrieveCellsByLocation(data_CSM_TH179NAT, sites_TH179NAT)$Mut

  data_census_fraction_sample[i, "NAT"] <- length(intersect(cells_TH179_NAT_normal, cells_TH179NAT))
}
}

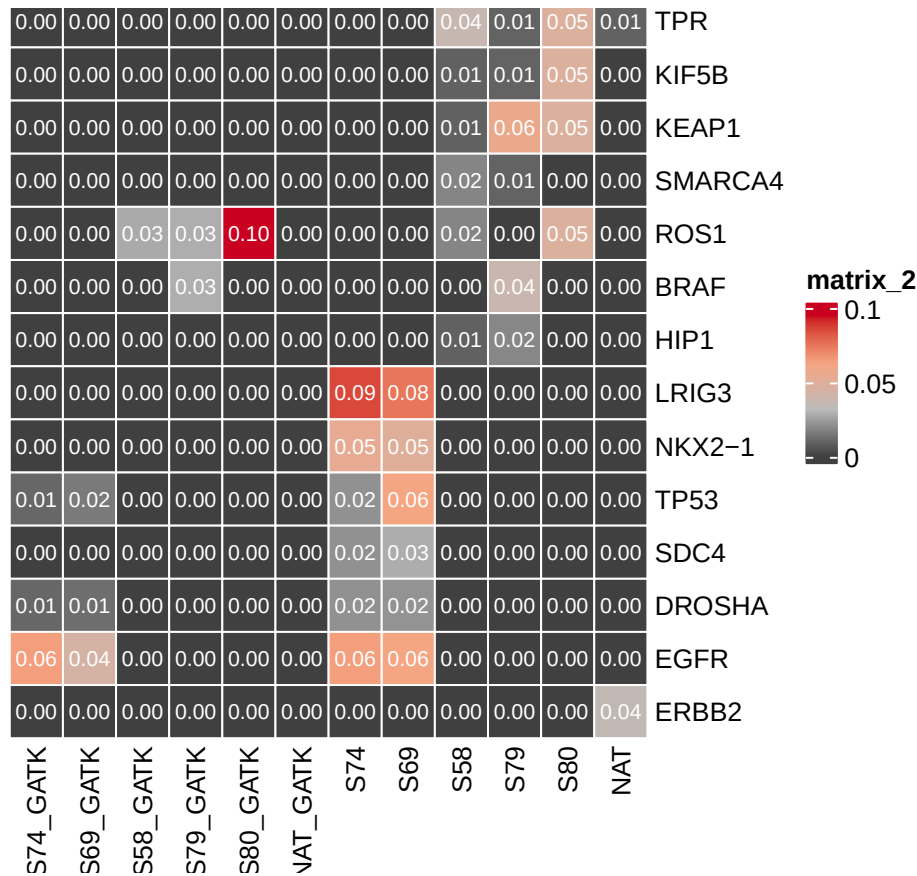
max_val <- max(data_census_fraction_sample)

col_fun_v1 <- colorRamp2(c(0, max_val * 0.33, max_val * 0.66, max_val), c("#404040", "#bababa", "#f4a582", "#f4a582"))

p_proportion_sample <- pheatmap(data_census_fraction_sample,
  cluster_rows = FALSE,
  cluster_cols = FALSE,
  cellwidth = 20,
  cellheight = 20,
  display_numbers = TRUE,
  number_color = "white",
  border_color = "white",
  color = col_fun_v1)
## Warning: The input is a data frame, convert it to the matrix.

p_proportion_sample

```



8 GermlinePopulation

```
# data_populationData_NSCLC.rds was generated using https://github.com/QunATCG/scMutrace-tutorial/tree/
data_population <- readRDS("../Data/HumanNSCLC/data_populationData_NSCLC.rds")
data_1kgenome_ann <- read.table("../Data/HumanNSCLC/1000genomeSampleInfo.txt", header = T, sep = "\t")
TH179_population_count <- data_population$TH179Count
TH179_population_fraq <- data_population$TH179Fraq

TH179NAT_population_count <- data_population$TH179NATCount
TH179NAT_population_fraq <- data_population$TH179NATFraq

TH248_population_count <- data_population$Th248Count
TH248_population_fraq <- data_population$Th248Fraq

length(colnames(TH179_population_count))
## [1] 28
sum(colnames(TH179_population_count) == colnames(TH248_population_count))
## [1] 28
sum(colnames(TH179NAT_population_count) == colnames(TH248_population_count))
## [1] 28

meta_population <- setdiff(colnames(TH179_population_fraq), c("male", "female"))
```

```

sample_size <- 100
freq_threshold <- 0.3
iteration_times <- 5000

data_count_sexPopulation_TH179_resample <- as.data.frame(
  matrix(data = 0, nrow = iteration_times, ncol = length(meta_population))
)
colnames(data_count_sexPopulation_TH179_resample) <- meta_population

data_count_sexPopulation_TH179NAT_resample <- as.data.frame(
  matrix(data = 0, nrow = iteration_times, ncol = length(meta_population))
)
colnames(data_count_sexPopulation_TH179NAT_resample) <- meta_population

data_count_sexPopulation_TH248_resample <- as.data.frame(
  matrix(data = 0, nrow = iteration_times, ncol = length(meta_population))
)
colnames(data_count_sexPopulation_TH248_resample) <- meta_population

set.seed(2025)

for (i in 1:iteration_times){
  selected_nrows <- sample(1:nrow(TH179_population_freq), sample_size, replace = FALSE)
  selected_data <- TH179_population_freq[selected_nrows, meta_population]
  data_count_sexPopulation_TH179_resample[i, ] <- colSums(selected_data > freq_threshold)
}

for (i in 1:iteration_times){
  selected_nrows <- sample(1:nrow(TH179NAT_population_freq), sample_size, replace = FALSE)
  selected_data <- TH179NAT_population_freq[selected_nrows, meta_population]
  data_count_sexPopulation_TH179NAT_resample[i, ] <- colSums(selected_data > freq_threshold)
}

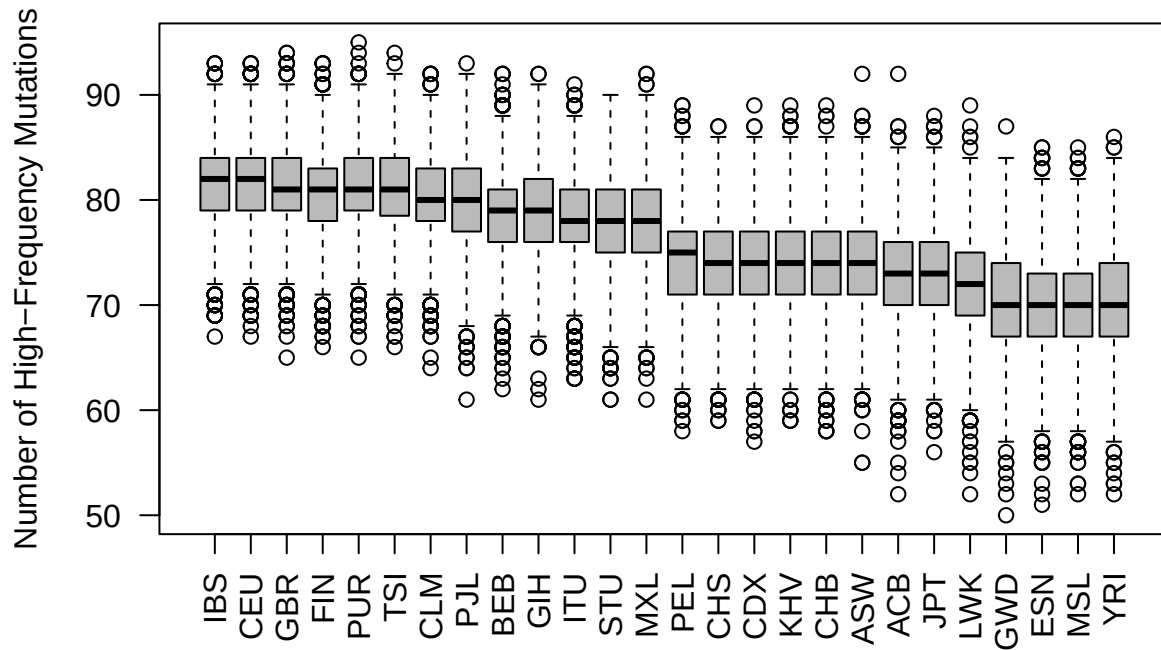
for (i in 1:iteration_times){
  selected_nrows <- sample(1:nrow(TH248_population_freq), sample_size, replace = FALSE)
  selected_data <- TH248_population_freq[selected_nrows, meta_population]
  data_count_sexPopulation_TH248_resample[i, ] <- colSums(selected_data > freq_threshold)
}

medians <- apply(data_count_sexPopulation_TH179_resample, 2, median)
sorted_cols <- names(sort(medians, decreasing = TRUE))

boxplot(data_count_sexPopulation_TH179_resample[, sorted_cols],
  las = 2,
  col = "#bababa",
  ylab = "Number of High-Frequency Mutations",
  main = "Resampled Mutation Frequencies Across Populations")

```

Resampled Mutation Frequencies Across Populations

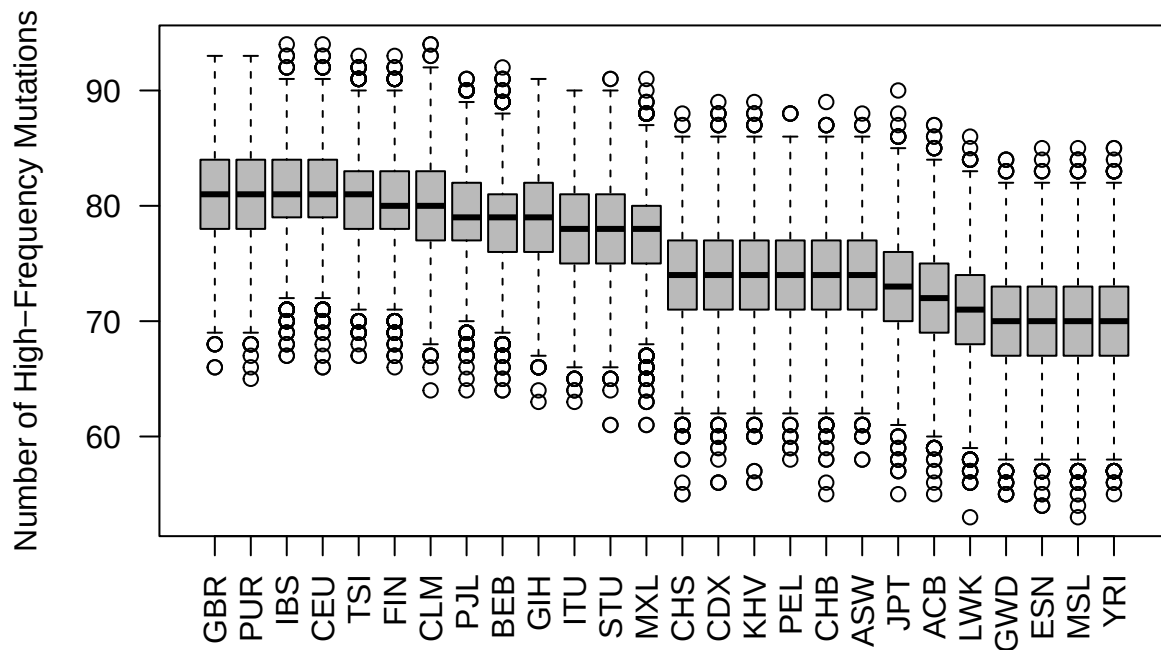


```
boxplot(data_count_sexPopulation_TH179_resample[, sorted_cols],
        las = 2,
        col = "#bababa",
        ylab = "Number of High-Frequency Mutations",
        main = "Resampled Mutation Frequencies Across Populations")

medians <- apply(data_count_sexPopulation_TH179NAT_resample, 2, median)
sorted_cols <- names(sort(medians, decreasing = TRUE))

boxplot(data_count_sexPopulation_TH179NAT_resample[, sorted_cols],
        las = 2,
        col = "#bababa",
        ylab = "Number of High-Frequency Mutations",
        main = "Resampled Mutation Frequencies Across Populations")
```

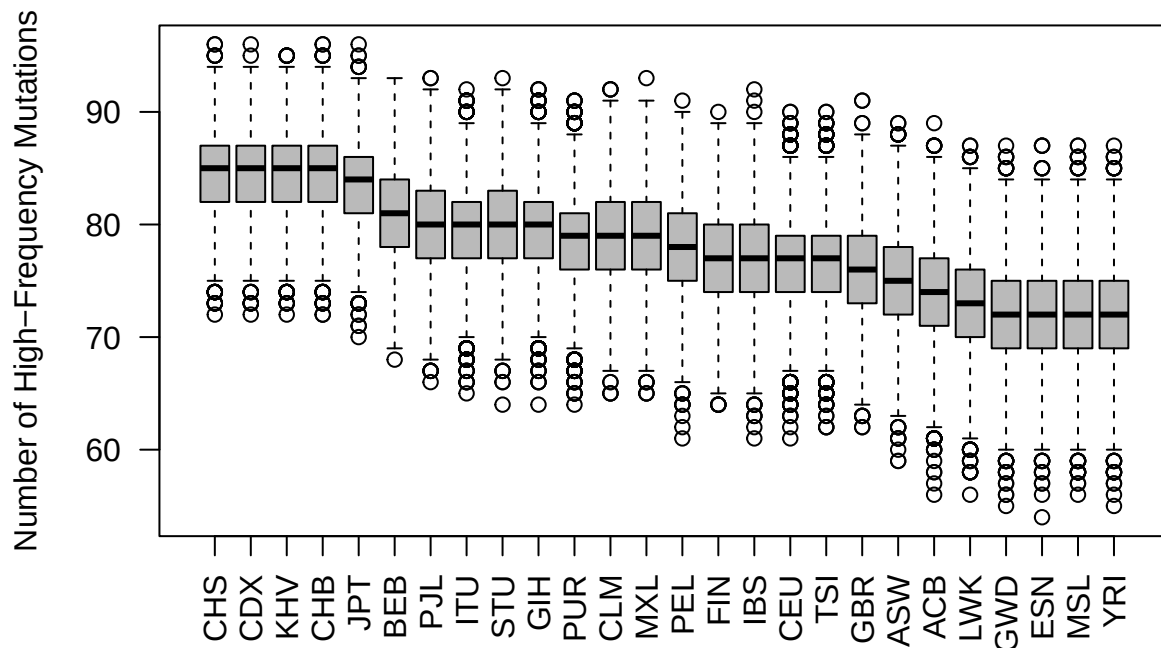
Resampled Mutation Frequencies Across Populations



```
boxplot(data_count_sexPopulation_TH179NAT_resample[, sorted_cols],
        las = 2,
        col = "#bababa",
        ylab = "Number of High-Frequency Mutations",
        main = "Resampled Mutation Frequencies Across Populations")

medians <- apply(data_count_sexPopulation_TH248_resample, 2, median)
sorted_cols <- names(sort(medians, decreasing = TRUE))
boxplot(data_count_sexPopulation_TH248_resample[, sorted_cols],
        las = 2,
        col = "#bababa",
        ylab = "Number of High-Frequency Mutations",
        main = "Resampled Mutation Frequencies Across Populations")
```

Resampled Mutation Frequencies Across Populations



```
boxplot(data_count_sexPopulation_TH248_resample[, sorted_cols],
  las = 2,
  col = "#bababa",
  ylab = "Number of High-Frequency Mutations",
  main = "Resampled Mutation Frequencies Across Populations")
```

9 sessionInfo

```
sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
```

```
##
## attached base packages:
## [1] grid      stats      graphics  grDevices utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] ComplexHeatmap_2.24.1 ggVennDiagram_1.5.4  circlize_0.4.16
## [4] dplyr_1.1.4          ggpubr_0.6.1         ggplot2_4.0.0
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6          shape_1.4.6.1        rjson_0.2.23
## [4] xfun_0.53             GlobalOptions_0.1.2  rstatix_0.7.2
## [7] Cairo_1.6-5          vctrs_0.6.5          tools_4.5.1
## [10] generics_0.1.4       stats4_4.5.1         parallel_4.5.1
## [13] tibble_3.3.0         cluster_2.1.8.1      pkgconfig_2.0.3
## [16] RColorBrewer_1.1-3    S7_0.2.0             S4Vectors_0.46.0
## [19] lifecycle_1.0.4      compiler_4.5.1       farver_2.1.2
## [22] codetools_0.2-20     carData_3.0-5        clue_0.3-66
## [25] htmltools_0.5.8.1    yaml_2.3.10          Formula_1.2-5
## [28] pillar_1.11.1        car_3.1-3            crayon_1.5.3
## [31] tidyr_1.3.1          iterators_1.0.14     abind_1.4-8
## [34] foreach_1.5.2        tidyselect_1.2.1     digest_0.6.37
## [37] purrr_1.1.0          labeling_0.4.3       fastmap_1.2.0
## [40] colorspace_2.1-2     cli_3.6.5            magrittr_2.0.4
## [43] dichromat_2.0-0.1    broom_1.0.10         withr_3.0.2
## [46] scales_1.4.0         backports_1.5.0      rmarkdown_2.30
## [49] matrixStats_1.5.0    ggsignif_0.6.4       png_0.1-8
## [52] GetoptLong_1.0.5     evaluate_1.0.5       knitr_1.50
## [55] IRanges_2.42.0       doParallel_1.0.17    rlang_1.1.6
## [58] glue_1.8.0           BiocGenerics_0.54.0  rstudioapi_0.17.1
## [61] R6_2.6.1
```